ML Performance Reading Group Notes

EleutherAI

1 Session 1: GPU Architecture, CUDA, NCCL

1.1 High-Level Summary

- **GPU Architecture**

> **Key Idea**
>
> **One-sentence difference:** CPUs are more latency-optimized and GPUs are more throughput-optimized.

So what is the difference between GPUs and CPUs?

In one sentence: CPUs are more latency optimized and GPUs are more throughput optimized.

- CPU = sports car
- GPU = bus

Example: Latency for GPU operations like register access and FMA, and L1/L2 access, can be much slower than on a CPU (but the GPU wins by running **many** things in parallel).

CPU architecture:

- A small number of cores, each with its own L1 cache and control logic attached.
- L2/L3 caches sit outside / above the cores (shared at higher levels).

GPU architecture:

- Many cores (lots of parallel lanes).
- L1 cache + shared memory close to execution units; L2 cache shared; DRAM = high bandwidth memory (HBM).
- Optimized for moving data fast and doing lots of arithmetic per unit time.

These cores exist in SMs (Streaming Multiprocessors). There might be a few to 100+ SMs depending on how big the GPU is.

There's a big variety of compute units on something like an A100/H100 (different ALUs for different floating-point types). GPUs also have specialized cores for matrix multiplication (Tensor Cores). There are register files that support fast context switching.

Key Idea

Warp concept (SIMT): A warp is an atomic scheduling unit of 32 threads. Threads in a warp execute the same instruction stream on different data (SIMT/SIMD-like behavior).

A GPU deals with warps as parallel groups of computation. Each SM has many warps; the warp scheduler rotates between them to hide memory access latency. While one warp waits for memory, the scheduler swaps to another warp so it hides the stall.

Divergence handling: If threads in the same warp take different control-flow paths (data-dependent logic), they serialize those paths and overall throughput drops.

Key Idea

Zooming out: whole GPU picture. In Ampere/Hopper-class systems:

- PCIe is the host interface (bus moving data between host and device).
- NVLink is a high-bandwidth interconnect between GPUs.

Latency comparison: memory vs compute (cleaned + preserved)

Speed breakdown (slow → fast):

$$\text{Global mem} < \text{L2} < \text{L1 / Shared} < \text{Registers} < \text{FMA} < \text{Tensor Cores}$$

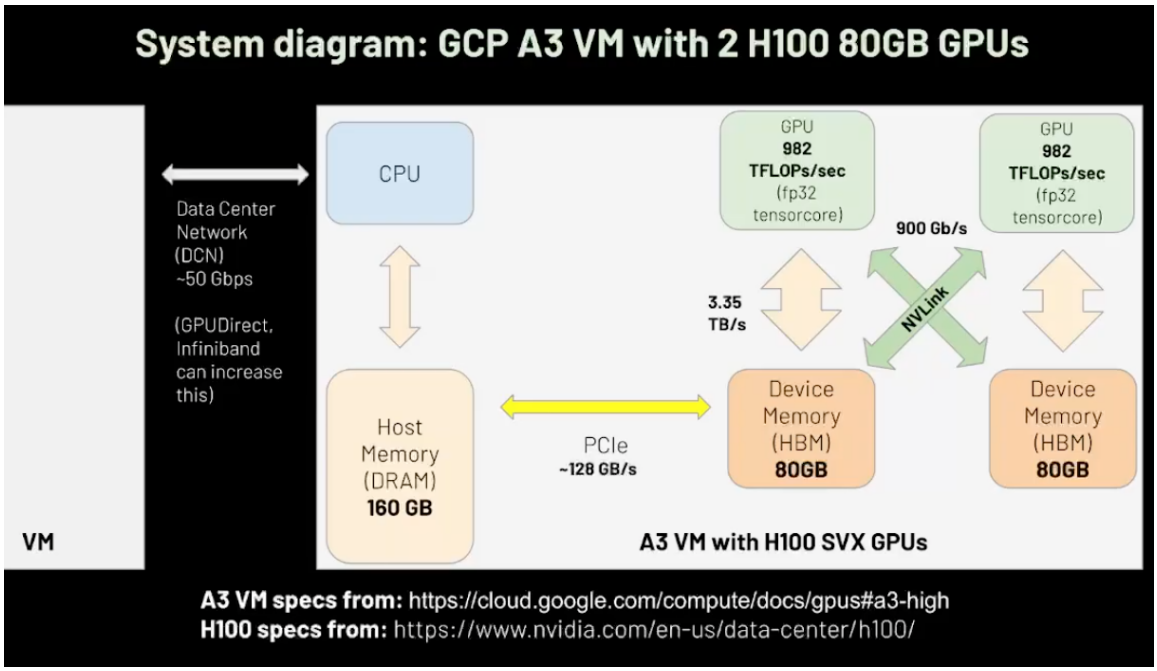
All we need to do is hide **memory latency** or **network communication** (those are often the slow parts).

Important

Performance intuition: GPUs usually have way more compute than memory bandwidth. If you can't feed the compute fast enough, compute units go idle.

System diagram (INSERT filled with image placeholder)

System Diagram



When doing massive distributed training runs on 1000s of GPUs and 100s of hosts, this is what we are working with.

Host/device are connected to the data center network (DCN) at around 50 Gbps. If you want to increase this connection speed you use GPUDirect or InfiniBand.

Above is an example of a VM with 2 H100s.

- Bandwidth for PCIe is 128 GB/s.
- Moving to GPUs:
  - Device memory is 80 GB.
  - Between device memory (HBM) and SRAM there is 3.35 TB/s.
  - NVLink between both GPUs is 900 GB/s.

The bottleneck here is the connection between the data center hosts. If you want to send to a neighboring host, data often needs to go:

$$\text{device} \to \text{host} \to \text{NIC / buffers} \to \text{DCN}$$

(This part is usually the bottleneck.)

Ways to combat this: RDMA (skip host copies by allowing the GPU to talk to the NIC directly, avoiding extra copying).

Now the compute contrast on a GPU:

- H100 uses FP32/TF32/etc. and does matmul on Tensor Cores at massive rates (compute is huge).
- This number looks inconsistent with data movement because you cannot feed compute fast enough unless you have techniques to cover that latency.

One solution in the compiler:

- Kernel fusion → more FLOPs per data that arrives from HBM (better reuse, fewer reads/writes).

FLOPs achievable and the data movement imbalance grows as we use lower precision data types like FP16 or INT8.

Key Idea

Factory analogy (preserved + cleaned):
Think of the GPU as a factory and memory/storage as the warehouse.
If the conveyor belt (bandwidth) is saturated, you get suboptimal throughput even if the factory (compute) is fast.

VOCAB (preserved + clarified):

- Arithmetic intensity = FLOPs / bytes moved over HBM (device memory).
  Compute it via profiling, compare to hardware ratio:
  - If workload ratio $<$ hardware ratio $\Rightarrow$ not compute-bound (likely memory or network-bound).
  - If workload ratio $\approx$ hardware ratio $\Rightarrow$ likely compute-bound (need better kernels or more GPUs).
- MFU (Model FLOPs Utilization) = value between 0 and 1: actual FLOPs achieved / achievable FLOPs on hardware.

Quick reference tables (added for memory recall)

Concept	CPU (sports car)	GPU (bus)
Optimization target	Latency	Throughput
Parallelism	Low–medium	Very high
Best at	Branchy/control-heavy code	Dense math + parallel workloads
Bottlenecks	Single-thread latency	Memory bandwidth + communication

Link	What it connects	Why it matters
PCIe	CPU/host ↔ GPU	Host-device bandwidth/latency
NVLink	GPU ↔ GPU	High BW multi-GPU on-node
InfiniBand/RDMA	Node ↔ node	Faster distributed training comms
GPUDirect RDMA	GPU ↔ NIC	Avoid host copies (lower latency)

- **CUDA**

  How do we make use of this highly parallel throughput machine?

  CUDA uses `nvcc` compiler and provides a set of software abstractions: a parallel model to organize computations and map them to hardware (breaking them into blocks/warps).

> **Key Idea**
>
> **CUDA mapping (fast recall):**
> **Grid** → many blocks across the GPU **Block** → runs on one SM **Warp** → 32 threads scheduled together

Kernel + matmul mapping (preserved + cleaned)

Kernel → (example) output matrix.

Let's say you have an output matrix:

- (1) Divide it into thread blocks (dimensions = how many threads tall and wide: `blockDim.y`, `blockDim.x`).
- (2) Once thread block is defined, the output has to be broken into the same blocks.

`gridDim.x, gridDim.y` = how many blocks wide/tall
`threadIdx.x, threadIdx.y` = each thread's local coordinates inside the block

> **Important**
>
> **INSERT (filled): naive matmul CUDA code (educational baseline)**
> This is the simplest version (not optimized). Real performance comes from tiling + shared memory + Tensor Cores.

Listing 1: Naive CUDA matmul (each thread computes one C element)

```
__global__ void matmul_naive(const float* A, const float* B, float* C,
                             int M, int K, int N) {
    // A: MxK, B: KxN, C: MxN
    int row = blockIdx.y * blockDim.y + threadIdx.y; // i
    int col = blockIdx.x * blockDim.x + threadIdx.x; // j

    if (row < M && col < N) {
        float acc = 0.0f;
        for (int k = 0; k < K; k++) {
            acc += A[row * K + k] * B[k * N + col];
        }
        C[row * N + col] = acc;
    }
}
```

Simple CUDA program outline (preserved + cleaned)

- (1) Define input matrices A and B, sizes $M \times K$ and $K \times N$.
- (2) Matmul output: C, size $M \times N$.
- Divide output matrix into thread blocks.
- Use ceiling/floor division so there are always enough blocks to cover the full output matrix.
- Each thread computes one element in C (dot product).
- Kernel launch uses `<<<grid, block>>>` to map computation to hardware.

CPU vs GPU implementation (preserved + clarified):

- CPU: loops compute dot products and write into C; uses strides to index arrays.
- GPU: compute (i, j) for C using block index + thread index:

  $i = \texttt{blockIdx.y} \cdot \texttt{blockDim.y} + \texttt{threadIdx.y}, \quad j = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}$

When we run a benchmark on the program, we see a large speedup on big matrices.

- **NCCL**

> **Key Idea**
>
> **NCCL:** a collective communication library → distributed compute primitives for multi-GPU ops.

Example: broadcast data to multiple GPUs, or do a reduction across GPUs. It gives tools to scale from one GPU to many GPUs/nodes.

Collective ops (preserved + cleaned + table added)

Rank is a unique identifier for a GPU (e.g., one node broadcasts weights to all others).

Collective	What it does (your wording, clarified)	Common use
Broadcast	One rank sends data to all other ranks	Send weights/config
AllReduce	Reduce across all ranks and store result in every rank buffer	Gradient sync (DP)
Reduce	Reduce across ranks, result stored in **one** rank	Aggregation
AllGather	Gather shards so every rank ends with full data	Unshard (FSDP)
ReduceScatter	Reduce, then scatter shards so each rank keeps only a shard	FSDP grads

Used for gradient sync: data-parallel training. Each GPU has a different model replica, runs a separate mini-batch, forward + backward, produces partial grads, then sync grads and perform the weight update.

AllGather (preserved + clarified): used when data is sharded across multiple devices (e.g., a layer split across devices). Before forward pass or for certain steps, you may need to unshard so everything is available.

ReduceScatter (preserved + clarified): instead of every rank having a full copy, each rank only has a shard of the aggregated data. For example, each device has partial grads; do reduce-scatter so each device keeps a shard.

Then after the update, you can do **AllGather** again (common in FSDP workflows).

2 Session 2: FlashAttention

Important

Key idea (preserved): be I/O aware!
Often it is faster to recompute than to load from HBM. Increase arithmetic intensity.

Motivation: long context is really hard with quadratic attention. Many heads and blocks. Each head must store, load, and compute huge matrices: you can run out of device memory, or need heavy offloading.

Arithmetic intensity = FLOPs / memory access.

More compute is available than bandwidth. When you load something, you don't want to do more saves than necessary. Crank arithmetic intensity up until you are compute-bound.

Standard attention device work (preserved + cleaned)

Device work:

1. Load blocks of query and key matrices onto compute from HBM.
2. Softmax.
3. Write back.
4. Read again.
5. Multiply by value matrix.

Often we do `torch.matmul` and everything "looks easy," but on GPU it has to tile it and decompose operations onto tensor cores.

On-chip view + tiling (preserved + clarified)

On chip: query and keys are loaded, softmax is applied. But softmax needs the entire row for stability—bad for long context (32k/64k).

Solution: tiling. GPUs do many small tile-wise accumulations:

- Load some tiles into SRAM.
- Compute smaller matmul accumulations (Tensor Cores).
- Accumulate partial results.

Key Idea

Tensor Cores (preserved + clarified):
Specialized hardware for dense matrix operations. Most peak FLOPs come from Tensor Cores.
If you do dense ops without Tensor Cores, performance will be much lower.

Tensor Core intuition: it operates on small blocks (e.g., 16×16 fragments).

Softmax stability (INSERT filled)

INSERT Softmax (x_i) formula:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Why subtract max?

$$\text{softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum_j e^{x_j - \max(x)}}$$

Subtracting $\max(x)$ improves numerical stability (prevents overflow), without changing the result.

Online / tiled softmax idea (preserved + clarified)

We can take separate softmax tiles, but eventually we need the whole row. While tiling, we track:

- running max (m)
- running denominator (ℓ)

When we update the max, we rescale numerator/denominator so we never materialize huge e^x values.

FlashAttention algorithm (filled with a faithful outline)

Below is a clean outline (the "shape" of the algorithm you described). You can replace variable names with the paper's exact notation later:

Key Idea

FlashAttention outline (tile-wise):
Process keys/values in tiles, maintain running softmax stats, accumulate output in SRAM.

Listing 2: FlashAttention-style tiled attention (high-level pseudocode)

```
# Inputs: Q [B,H,seq,d], K [B,H,seq,d], V [B,H,seq,d]
# Output: O [B,H,seq,d]

for each query tile Qt:
    m = -inf            # running max per row
    l = 0               # running denom per row
    O_acc = 0           # running output accumulator
```

```
    for each key/value tile (Kt, Vt):
        S = Qt @ Kt.T              # scores tile
        S = S * scale              # scale
        S = apply_causal_mask(S)   # if causal

        m_new = max(m, rowmax(S))
        P = exp(S - m_new)         # stable exp for this tile

        l = l * exp(m - m_new) + rowsum(P)
        O_acc = O_acc * exp(m - m_new) + (P @ Vt)

        m = m_new

    O = O_acc / l                  # normalize at end
```

Key thing: based on SRAM sizes we choose block sizes so tiles fit in fast memory. Then we do normal tiling computation, maintain m and ℓ, and do final rescaling. Because the math is associative after rescaling, we can change operation order to be faster.

This process cut down training time significantly, especially for long context.

FlashAttention 2 and 3 (preserved + cleaned)

FlashAttention 2: reduced many redundant scaling operations.

- Tensor Cores are so fast that pointwise ops in SRAM are relatively cheap.
- Swap loop order to allow better parallelization of Q tiles; long context often means low batch size, so better residency over hardware matters.

FlashAttention 3: uses fancy async features of H100:

- async memory ops
- Tensor Core batching/pipelining

Triton kernel walkthrough (preserved + cleaned, kept long on purpose)

Just to go over the Triton kernel implementation for basic optimization:

Key Idea

This is a simpler, intuitive implementation of FlashAttention written in Triton. The goal is to reduce memory traffic while computing attention by tiling Q/K/V blocks and performing an online softmax accumulation.

Simple intuitive implementation of autograd (forward pass focus):
Starts with the forward pass.
First thing: define the **grid** (how computation is parallelized on the hardware).

Important

In Triton:

- A **program instance** ≈ CUDA thread block (not individual threads).
- Each program handles a tile of data (here: a block of query vectors).
- Parallelization typically spans:
  - query sequence blocks
  - batch dimension
  - attention heads

We divide sequence length by block size for Q. Each element corresponds to a query vector, then we chunk vectors into Q blocks.

So effectively:

- Parallelize across queries
- Across batches
- Across attention heads

Kernel launch setup

Using Triton to launch the kernel:

We pass strides from PyTorch tensors. These help index memory correctly when loading tiled blocks.

Key Idea

Strides allow navigation of multidimensional tensors without copying memory — critical for efficient block pointer arithmetic.

Inside the kernel

We determine which program instance we are:

- Query block index
- Batch index
- Head index

Batch and head indices are often packed together and later decomposed to identify the exact attention head.

Then:

- Move from base Q pointer to the correct subtensor offset
- Use strides for accurate indexing

Block pointer construction
We create a block pointer:

Key Idea

A block pointer represents a 2D tile of memory that Triton loads efficiently from HBM (global GPU memory) into on-chip SRAM/cache.

Once we have the parent subtensor:

- Extract Q block of size BLOCK_Q
- Apply offsets for correct row selection

Handling K and V
KV follows similar logic.
Important detail:

Important

K is effectively transposed relative to Q in the attention matmul:

$$QK^T$$

This is why access patterns differ slightly.

We iterate over K/V blocks sequentially along the sequence axis.
Online softmax variables
After pointer arithmetic:

- m_i = running row max (numerical stability)
- ℓ_i = running softmax denominator

This enables memory-efficient FlashAttention-style accumulation.
SRAM loading + inner loop
First:
Load Q block into SRAM (fast on-chip memory).
Then inner loop:

- Iterate over K/V blocks sequentially
- For causal attention:
  - Only attend to previous tokens
  - Stop at diagonal boundary

Masking:

- Use row/column offsets
- Compare indices for causal mask

Then pipeline:

1. Load K/V block (HBM → SRAM)
2. Compute matmul
3. Apply scaling
4. Apply causal mask

Online softmax update (FlashAttention core)
Steps:

- Compute local row max m_i
- Compute correction factor:
$$\exp(m_{\text{prev}} - m_{\text{local}})$$
- Adjust previous denominator because of tiling

Then:

- Compute probability block P
- Compute row sum (partial denominator)

New global denominator:

$$\ell_{\text{new}} = \text{correction factor} \cdot \ell_{\text{prev}} + \text{row sum}$$

Update global max:

$$m_i \leftarrow \max(m_{\text{prev}}, m_{\text{local}})$$

Output accumulation
Update output block O:

- Scale previous output by correction factor
- Add new matmul contribution with V

This avoids storing the full attention matrix.
Advancing KV blocks
Iterate to next key/value blocks:

- K pointer advances along sequence dimension
- V pointer advances similarly

This continues until all KV blocks are processed for the current Q block.
Final writeback
After all K/V blocks:

- Normalize output by final denominator

- Write output block back to HBM

Important

Key optimization idea:
Never materialize full attention matrix in memory. Everything is streamed block-by-block with online softmax accumulation.

3 Session 3: ZeRO (Zero Redundancy Optimizer)

Key Idea

High-level idea :
ZeRO (Zero Redundancy Optimizer) reduces GPU memory by removing redundancy in **data-parallel training**. It does this by partitioning (sharding) model states across GPUs instead of replicating them on every device.

3.1 Motivation: the scaling problem (why ZeRO exists)

Models are getting too big for a single process or a single device — the memory footprint grows quickly as the number of parameters and layers increase.

Example (your note, clarified): BERT with $\sim$ 320M parameters is already large enough that memory can become a bottleneck depending on precision, batch size, sequence length, and optimizer.

Important

What actually blows up memory?
There are two major buckets:

- **Steady-state memory:** parameters (weights), gradients, and optimizer states.
- **Peak memory:** activations saved for backward (often grows with batch size and sequence length).

Even if steady-state fits, peak activation memory can still OOM you.

3.2 Steady-state memory breakdown

Let N be the number of parameters.

For **single-precision (FP32)** training (common baseline):

- **Parameters:** $4N$ bytes
- **Gradients:** $4N$ bytes
- **Optimizer states (Adam-like):** typically two FP32 moment buffers $\Rightarrow$ about $8N$ bytes

So a common "steady-state" approximation for Adam-style training is:

$$\text{model states} \approx 4N \text{ (params)} + 4N \text{ (grads)} + 8N \text{ (optimizer)} = 16N \text{ bytes.}$$

Key Idea

Key correction (fills the misconception):
Optimizer states are not "double the size of weights+grads+params combined."
For Adam, optimizer states are typically about $2\times$ the parameter size (two moment buffers), so the **optimizer** is often the largest single chunk, but the total model-state memory is commonly approximated as $\sim 16N$ bytes in FP32 training.

Concrete example: 320M parameters (filled + corrected)

Let $N = 320\text{M}$.

$$4N = 4 \cdot 320\text{M bytes} = 1280\text{MB} \approx 1.28\text{GB}.$$

Component	**Bytes**	**Approx size (for $N = 320$M)**
Parameters (FP32)	$4N$	≈ 1.28 GB
Gradients (FP32)	$4N$	≈ 1.28 GB
Adam optimizer states (m, v in FP32)	$8N$	≈ 2.56 GB
Total steady-state (FP32 + Adam)	$16N$	≈ 5.12 GB

Important

Note :
Input data memory is usually not the main issue compared to activations and model states.
Peak memory is often dominated by activations: layernorm + attention block + FFN + loss (and anything saved for backward), and it can grow significantly as batch size / sequence length increase.

3.3 Precursor solutions to reduce memory footprint

1) Mixed precision (FP16/BF16 + FP32 master weights)

Mixed precision uses half precision for most compute and activation storage, while keeping a FP32 "master copy" of weights for stable updates.

- Do forward/backward primarily in FP16/BF16 (faster, less memory).
- Keep FP32 master weights for optimizer updates (stability).
- Activation storage often decreases a lot, which helps peak memory.

- Use loss scaling to prevent gradient underflow in FP16.

Key Idea

Why memory can still go down even with "two copies" of weights:
The big savings often come from activations (peak memory) and sometimes gradients/comm buffers, not just from the raw weight tensor. Keeping FP32 master weights helps quality/stability.

2) Data parallelism (DP) (baseline that ZeRO builds on)

In data parallelism, the input data batch is split across devices/processes. Each device runs forward+backward on its mini-batch independently.

- **Pro:** simple scaling; no pipeline bubbles; independent execution.
- **Con:** each device stores a full copy of model states (params + grads + optimizer), so memory footprint is not reduced.
- Gradient synchronization happens via a collective at the end (commonly AllReduce or ReduceScatter + AllGather depending on implementation).

Important

This is the key inefficiency ZeRO targets:
DP scales well, but it is memory-inefficient because model states are **replicated** on every GPU.

3) Pipeline parallelism (layer-wise split)

Pipeline parallelism splits the model by layers across devices. This can be memory efficient since a device only stores the layers it owns, but scheduling is required to reduce idle time ("bubbles").

- Potential issue: stages can sit idle due to dependencies (a pipeline bubble).
- Mitigation: micro-batching (pipeline multiple micro-batches so later stages stay busy).
- "Zero-bubble" scheduling strategies exist (not perfectly zero, but reduces idle time).

4) Tensor parallelism (split tensors within a layer)

Tensor parallelism splits the computation within layers across devices (e.g., splitting matmul weight matrices).

- **Pro:** reduces per-device compute and per-device parameter shard size for that layer.
- **Con:** requires communication between devices (can block).
- Common collectives involved: AllReduce, AllGather, ReduceScatter.

Important

Row-wise vs column-wise split :

- **Column-parallel linear (split output features):** each GPU computes a slice of the output. To feed the next layer, you often need to AllGather outputs (or keep the next layer sharded compatibly). Backward typically involves communication to combine gradients.
- **Row-parallel linear (split input features):** each GPU consumes a slice of the input. Forward often requires the input to be available in sharded form, and you typically AllReduce partial outputs (sum contributions) so downstream sees the full result.

Different frameworks fuse these patterns so that the AllGather/AllReduce happens at the cheapest points.

5) CPU offloading (preserved + cleaned)

Offloading moves some tensors (optimizer state, parameters, activations, or checkpoints) to CPU memory.

- **Pro:** leverages large host memory; useful for checkpointing and staging.
- **Con:** CPU throughput is lower and PCIe can become the bottleneck if you offload too aggressively.

6) Sharding (preview)

Sharding means partitioning tensors across devices so each GPU holds only a fraction, reducing redundancy. ZeRO applies this idea specifically to model states.

3.4 Why ZeRO builds on data parallelism (preserved + clarified)

Your core reasoning is right: DP has strong scaling properties, but is memory-inefficient.

1. **DP scales well:** each rank does full forward/backward on its data shard (self-contained compute). Communication is mostly around gradients, which is predictable and can be optimized.
2. **DP is memory-inefficient:** parameters/gradients/optimizer states are replicated on every GPU.
3. **Model parallelism limits:** pipeline/tensor parallelism are often limited by how many accelerators you have per node; scaling beyond a node requires inter-node communication (usually lower bandwidth/higher latency than on-node).

Key Idea

ZeRO's pitch (one line):
Keep DP's nice scaling behavior, but remove DP's redundant memory replication by partitioning model states.

3.5 ZeRO stages (the core of Session 3)

ZeRO partitions the DP "model states" in stages:

- Optimizer states (largest)
- Gradients
- Parameters

Stage	What is partitioned (sharded) across GPUs	Main effect (your idea, clarified)
ZeRO-1	Optimizer states	Big steady-state memory reduction (optimizer often dominates)
ZeRO-2	Optimizer states + gradients	Further steady-state reduction; gradients reduced/scattered instead of replicated
ZeRO-3	Optimizer states + gradients + parameters	Maximum steady-state reduction, but more communication (params must be gathered when needed)

Stage 1 (optimizer partitioning)

- Optimizer state is partitioned across devices.
- Each device updates the weights corresponding to its optimizer partition.
- Parameters/gradients may still be replicated depending on configuration, but you get a large win because optimizer state is usually huge.

Stage 2 (gradient partitioning)

- Partition gradients across devices.
- Use ReduceScatter to aggregate and distribute gradient shards.
- Each device only keeps (and applies) the gradient shard needed for its optimizer shard.

Stage 3 (parameter partitioning)

- Partition parameters across devices (each device holds only a shard at rest).
- During forward/backward, layers AllGather parameter shards just-in-time, then release when done.
- Tradeoff: more communication than baseline DP because parameters are not always resident locally.

Important

Key idea (preserved):
ZeRO never materializes the full redundant model state on every GPU. Instead, it streams/shards what is needed, when it is needed, and frees it afterward.

3.6 ZeRO-R and activation/peak memory

Activation memory is often the limiting factor for batch size. ZeRO-R focuses on reducing peak memory by addressing activation-related issues.

Two ideas you mentioned (cleaned):

- **Activation checkpointing / recomputation:** save fewer activations; recompute them during backward to save memory.
- **Memory fragmentation:** activations are short-lived while gradients/parameters are longer-lived; interleaving these can fragment memory. Better memory management/checkpointing patterns help.

Communication notes

Baseline DP communication is largely dominated by gradient synchronization (often implemented as ReduceScatter + AllGather, which together behave like an AllReduce).

ZeRO stage 3 increases communication because parameters must be gathered (AllGather) when needed. ZeRO-R can add additional AllGather-style ops depending on how activations/checkpoints are partitioned and recomputed.

3.7 Other variants you listed

- **ZeRO-Offload:** offload optimizer state / parameters (and sometimes gradients) to CPU memory. Great for capacity, but must be careful not to bottleneck on PCIe.
- **Quantized gradients:** compress gradients to reduce communication volume (replaces/augments standard gradient collectives).
- **Quantized weights:** reduce parameter AllGather volume (e.g., FP16 $\rightarrow$ INT8) to cut comm costs in ZeRO-3-like setups.

3.8 Compatibility with other parallelism

- ZeRO was designed to work naturally with data parallelism.
- It can be combined with tensor parallelism in practice (common in large-model stacks).
- Combining ZeRO-2/3 with pipeline parallelism is possible but can be trickier to keep efficient because sharded gradients/parameters must align with pipeline scheduling.

3.9 ZeRO vs PyTorch FSDP

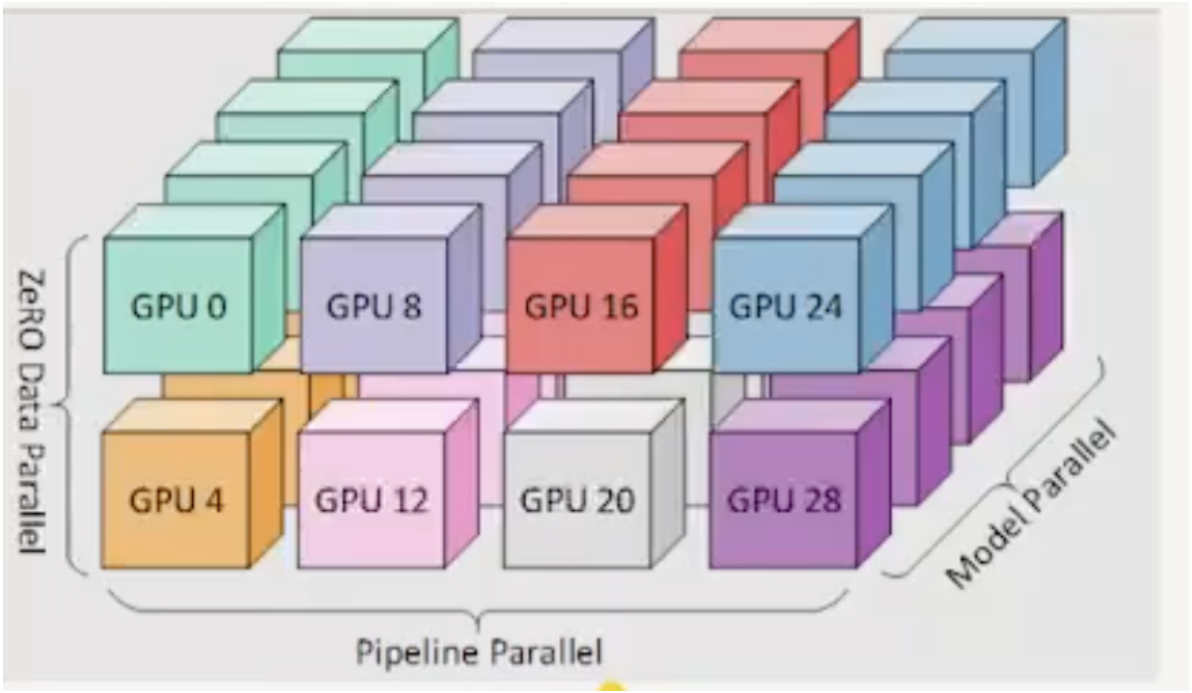
FSDP (Fully Sharded Data Parallel) is PyTorch's implementation of ZeRO-style sharding. Conceptually:

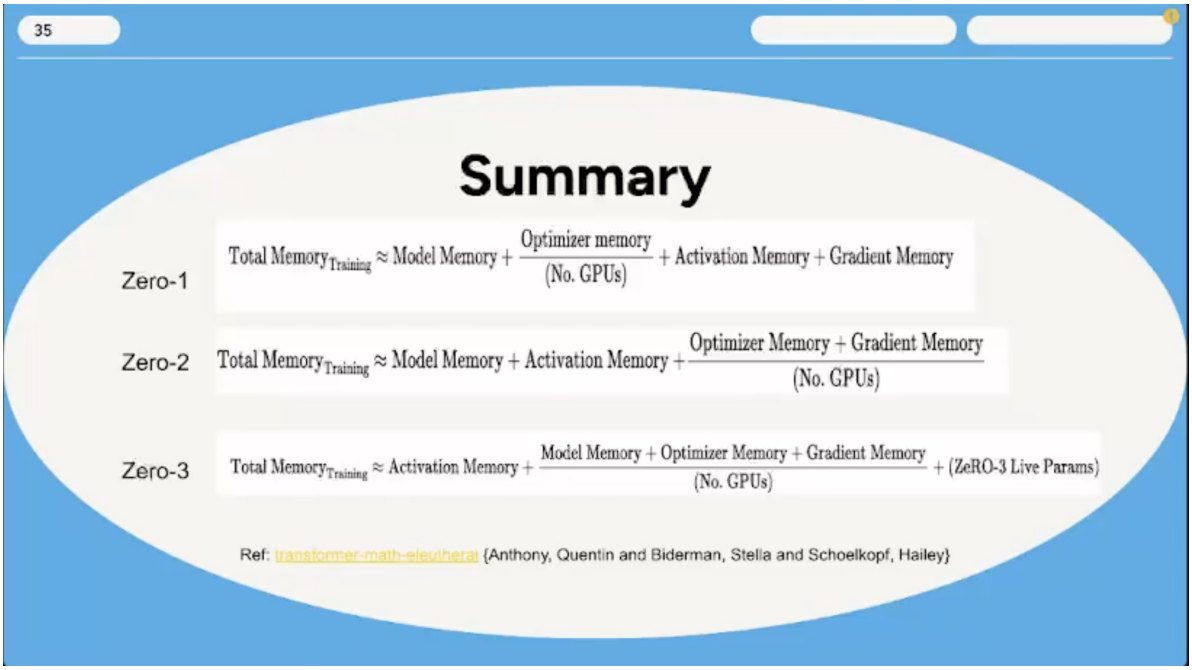
- Both shard parameters (and often grads/optimizer states) across ranks.
- Different configuration knobs in FSDP correspond closely to ZeRO stages (what to shard, when to gather, and what to offload).

Figures

Important

Key diagrams (for recall): the first shows the ZeRO stage idea (redundancy removal / partitioning), and the second summarizes memory savings across stages.





4 Session 4: Ring Attention

Important

Session framing (preserved + cleaned): Ring self-attention v1 did not show a huge impact in all settings, but later versions (combined with blockwise parallel ideas) showed stronger memory and communication benefits.

4.1 Scope (three papers/topics you listed)

1. **Sequence Parallelism (Ring Self-Attention)**
2. **Blockwise Parallel Transformers**
3. **Third paper/topic (placeholder in your notes)**

4.2 Why this line of work exists

- Long-context attention creates a major memory bottleneck.
- Even with tensor/pipeline parallelism, very large sequence lengths remain expensive.
- Goal: increase maximum context length while keeping communication and memory manageable.

4.3 Sequence Parallelism via Ring Self-Attention (preserved + clarified)

Key Idea

Core mechanism: Partition the sequence dimension across devices. Keep each device's local query block fixed, and rotate key/value blocks in a ring so each device can compute full attention output for its local queries.

Execution sketch (your idea, organized):

1. Partition tokens across devices.
2. Compute local Q/K/V projections for each token shard.
3. For each device, keep local query block fixed.
4. Rotate key blocks around the ring:
   - At each step, compute partial attention scores for current key block.
   - Overlap compute with send/receive of next key block.
5. After $N-1$ ring steps (for N devices), each device has seen all key blocks for its local queries.
6. Rotate value blocks similarly and accumulate final attention outputs.

Important

Misconception cleared: This is not linear-attention approximation. Ring attention still computes **exact attention** for each query; it changes **partitioning and communication pattern** to reduce per-device memory pressure and improve scalability.

4.4 Complexity and scaling intuition (cleaned from your notes)

- Global attention remains quadratic in sequence length overall.
- Per-device memory/working set is reduced via sequence partitioning.
- Communication is structured ring exchange instead of full all-gather of all keys/values at once.
- This can enable much longer contexts in practice (your note: up to $\sim$ 114k tokens in reported setups).

Approach	What is partitioned	Main implication
Tensor parallelism (baseline in your note)	Model weight tensors/matmul dimensions	Requires synchronization/all-gather patterns around sharded matmuls
Sequence parallelism (ring attention)	Token/sequence dimension	Keeps local query block fixed, rotates K/V blocks, lowers per-device sequence memory load

4.5 Batch-size observation from your notes

- You noted better efficiency when batch size is larger.
- In your notes: multi-head attention with sequence parallelism looked more efficient for batch size > 16 in the studied setting.

4.6 Blockwise Parallel Transformers (placeholder preserved)

- Your note: this was a key reason later versions had stronger impact.
- Working interpretation: blockwise execution helps sustain lower memory footprint and communication overhead.
- Keep this section as a placeholder until paper-specific equations/algorithms are added.

4.7 Third paper/topic (placeholder preserved)

- Placeholder retained from original notes.
- Add title + key contribution once finalized.

5 Session 8: Megatron-LM – Training Multi-Billion-Parameter Models with Model Parallelism

Key Idea

Session 8 thesis (your notes, organized):
Megatron-LM combines tensor model parallelism (intra-layer splits) with pipeline/data-parallel ideas to scale large Transformers while controlling communication overhead.

5.1 Why this paper mattered

- The paper demonstrated strong scaling from smaller to much larger GPU counts.
- Your notes emphasize this practical result: good scaling efficiency is achievable, but communication overhead is the main bottleneck to manage.
- Goal: keep mathematically equivalent training while changing **where** and **when** communication occurs.

Important

Guiding rule (preserved + clarified):
Do not change model math; only change operation partitioning and synchronization timing.

5.2 Transformer block decomposition used in the notes

- Self-attention block
- MLP block (two GEMMs with GELU nonlinearity in between)
- Residual + dropout around sublayers

5.3 Key MLP insight: avoid synchronization before GELU

Your notes highlight a central design choice:

- If a split produces **partial sums** before a nonlinearity, applying GELU locally is not equivalent to applying GELU after global sum.
- Therefore, that split forces an early all-reduce, which hurts performance.
- Megatron chooses partitions that delay synchronization until after nonlinearity-safe points.

Important

Why early sync is bad here:
For nonlinear f, in general $f(a+b) \neq f(a)+f(b)$. So if each GPU only has partial pre-activation sums, you must communicate before GELU to preserve exact math.

MLP partition pattern (as reflected in your notes)

1. First projection GEMM: **column-parallel** split (each rank computes complete local outputs for its shard).
2. Apply GELU **locally** on each shard (safe, no immediate sync).
3. Second projection GEMM: **row-parallel** split.
4. All-reduce partial outputs at the end of the MLP block.

Key Idea

Recall line: first GEMM column split, second GEMM row split, then synchronize once at block output.

5.4 Self-attention tensor parallelism (from your notes)

- Exploit natural independence across attention heads.
- Replicate input activation X across tensor-parallel ranks.
- Shard Q/K/V projections by heads so each rank computes attention for its local head subset.
- Perform local attention (QK^T, softmax, PV) without intermediate cross-rank communication.
- Project back to hidden dimension and all-reduce at the output projection stage.

5.5 Communication pattern summary

Block	Local compute	Synchronization point
MLP	GEMM_1 + GELU + GEMM_2 (sharded)	All-reduce at block output
Self-attention	QKV + attention per local heads	All-reduce at output projection

Your note that this design uses a small number of all-reduces per Transformer layer (forward/backward) matches the core optimization target: fewer, well-placed synchronizations.

5.6 Input and output embedding parallelism

You captured an important memory point: the vocabulary embedding matrix is huge, so sharding by vocabulary dimension is beneficial.

Input embedding logic (filled + clarified)

- Split embedding table across vocabulary rows (each GPU owns a token-ID range).
- For each token ID:
  - If ID is in local shard range, perform local lookup.

  - Otherwise, contribute zeros (masked lookup).
- Sum partial embedding contributions across ranks (all-reduce or reduce-scatter/all-gather variants depending on the parallel stack).
- Result: every rank gets the correct full embedding activations while storing only a shard of the embedding table.

Output embedding / vocab logits logic (placeholder filled)

- Output projection is also vocabulary-sharded: each rank computes logits for its local vocab slice only.
- Naively, full softmax would require gathering all vocab logits.
- Megatron-style implementations avoid a full logits all-gather by fusing parallel logits with cross-entropy:
  - compute local max/log-sum-exp contributions,
  - reduce across ranks for global normalization terms,
  - compute loss/gradients directly from sharded logits.
- This significantly reduces communication and memory traffic compared with materializing full vocab logits on every rank.

Important

Misconception corrected: You usually do **not** need to all-gather the full vocabulary logits for cross-entropy in tensor-parallel training. Fused sharded cross-entropy computes the exact loss with reduced communication.

5.7 Practical takeaway (your notes distilled)

- Tensor model parallelism is most effective when communication stays mostly intra-node (high-bandwidth links).
- Main design challenge is not splitting compute, but placing synchronization so nonlinear correctness is preserved and communication is minimized.
- Vocabulary sharding and fused loss are crucial for scaling large-vocab LMs.

6 Session 16: LMCache

Key Idea

Session 16 thesis (from your notes, cleaned):
LMCache treats KV cache as a first-class system component and combines it with prefill/decode disaggregation and paged attention to improve end-to-end inference efficiency.

6.1 Motivation

- Processing each user request fully independently is inefficient, especially with system prompts, multi-turn context, and long traces.
- End-to-end serving has mixed phases: some are compute-bound (prefill), others are memory or communication-bound (decode).
- Core goal: reuse prior compute (KV cache), minimize redundant work, and improve GPU utilization plus first-token latency.

6.2 Why KV cache helps

Your original intuition, organized: modern LLMs are decoder-based and generate tokens autoregressively. You feed in a prompt, the model runs it through many Transformer layers, and then predicts the next token. That next token is appended to the sequence, and the process repeats.

- Each layer produces **query**, **key**, and **value** vectors.
- Attention uses queries against keys to produce scores, and then uses those scores to weight the values.
- For the newest token, the only query we need is the query for that newest token, but it must attend over **all previous keys and values**.
- Without caching, the system would keep recomputing old keys and values again and again at every decode step.

Added Clarification

Correction added:
KV cache stores **keys and values**, not queries. Query vectors are computed for the current token at decode time.

Redundant-work picture (preserved + clarified):

- Generating token 50 requires access to K/V for tokens $1 \ldots 50$.
- Generating token 51 requires access to K/V for tokens $1 \ldots 51$.
- If you recompute all prior K/V every step, you are repeating a huge amount of projection work.

So the fix is:

1. Compute Q/K/V for only the newest token.
2. Append the new K and V to the cache.
3. Retrieve the previously cached K/V for all earlier tokens.
4. Run attention using the new query against the full cached history.

Important

Important compute intuition:
KV caching removes repeated **projection work** for past tokens, but decode still scales with sequence length because each new token still attends over all cached history.

6.3 Prefill, decode, and TTFT

Your original reasoning chain:

- The first token is slow because the model must process the entire prompt first.
- That first full pass computes and stores K/V for every prompt token.
- This is the prefill phase, and it is the most compute-intensive part of the request.
- Once the cache is warm, later tokens only need a one-token decode step, so generation becomes much cheaper.

Phase	What happens	Why it matters
Prefill	Full prompt forward pass, build KV cache	Dominates first-token latency
Decode	One-token step using cached history	Cheaper per token, but memory-sensitive

TTFT (time to first token):

- Longer prompt ⇒ longer prefill.
- Longer prefill ⇒ longer wait before the first generated token.
- This is why TTFT optimization is its own topic: chunked prefill, speculative decoding, and prompt caching all target this initial delay.

Key Idea

Recall line:
Building the cache is expensive. Reading from the cache is cheap.

6.4 Memory tradeoff of KV cache

- KV caching trades compute savings for memory usage.
- Every layer stores K/V tensors for every token in the active context.
- As context length increases, KV cache grows linearly with sequence length.
- At high concurrency, KV memory can become one of the dominant serving costs.

Your original concern, preserved:

- For long-context models and many concurrent requests, KV cache can become so large that it rivals or exceeds model-weight memory.
- This is one reason MQA and GQA exist: they reduce the number of key/value heads that must be stored.
- Doubling context length is hard in practice partly because it also doubles per-request KV-cache footprint.

6.5 Prefill/decode disaggregation (PD disaggregation)

Your original system-level point: prefill and decode are very different workloads, so it often makes sense to split them.

- Prefill is dense-matrix heavy and benefits from high-throughput accelerators.
- Decode is token-by-token, latency-sensitive, and often bottlenecked by memory movement plus cache access.
- Splitting these phases across different workers/devices can improve throughput and cost-efficiency, depending on workload.
- The decode-side requirement is enough fast memory capacity/bandwidth to serve KV cache efficiently.

Added Clarification

Added nuance:
PD disaggregation is powerful for large or mixed workloads, but can be overkill for smaller models or low concurrency where transfer overhead dominates.

How you were thinking about it (preserved + cleaned):

- You do not want to waste the same GPU doing both heavy prompt processing and low-throughput token streaming if those phases want different hardware characteristics.
- Prefill can run on a large, throughput-oriented GPU.
- Decode may be cheaper to run on a device optimized for fast cache access and serving latency.
- LMCache makes this practical by letting prefill produce cache and decode consume it elsewhere.

6.6 Paged attention and cache paging

Your original motivation: systems do not want to move or allocate memory one token at a time if the transport and allocator both prefer chunked operations.

- Variable-length requests can cause memory fragmentation if KV memory is managed as large contiguous buffers.
- Paged attention uses fixed-size pages from a shared pool and maps each request to a list of pages.

- This improves reuse and flexibility: pages for shared prefixes (for example, common system prompts) can be reused.
- Data movement is performed in page-sized chunks instead of tiny per-token transfers.

Your original allocator intuition, organized:

- If every request gets a custom-sized contiguous buffer, memory becomes fragmented.
- Fixed-size pages are easier to reuse, move, and share.
- A request effectively becomes a list of pages rather than one giant flat tensor buffer.
- Shared system prompts are especially attractive because those pages may be reusable across many requests.

6.7 LMCache design philosophy (from your notes)

- Treat KV cache as a first-class object (move, pin, evict, compress, and tier it explicitly).
- I/O-aware operation: batch and size transfers near each device/link sweet spot.
- Modular interface: a common API that multiple inference engines can call.
- Explicit cache control enables better coordination with prefill/decode split pipelines.

6.8 System architecture

- **Cache controller:** API-facing control plane (router/scheduler interaction).
- **Worker process:** data plane that moves/manages cache across GPU memory and storage backends.
- This architecture supports distributed deployment across multiple hosts/instances.

6.9 Request lifecycle (organized from your steps)

Your original request flow, restored in order:

1. **Request arrival + metadata routing:** identify reusable cache and choose target worker/device.
2. **Page allocation plan:** reuse existing pages, allocate missing pages, prepare transfer plan.
3. **KV load + prefill overlap:** pipeline cache transfer and compute so later-layer cache can arrive while earlier layers run.
4. **Decode/token generation:** generate tokens incrementally while updating/transferring KV in batches.
5. **Response + cleanup:** return tokens, update cache metadata, evict or persist pages by policy.

Step-by-step interpretation from your notes:

- When a request arrives, the scheduler/router asks whether relevant cache already exists somewhere in the system.
- If cache exists, the system decides which worker/device is best positioned to use it.
- It allocates or reuses pages, then starts pulling the needed cache toward the compute path.
- Cache loading can overlap with prefill/compute so the next layer's data is arriving while the current layer is executing.
- During generation, KV updates are batched and transferred in page groups rather than tiny fragmented writes.
- After the request finishes, the system decides what to keep hot, what to evict, and what to push to colder tiers.

6.10 Recompute vs fetch decision

- A practical scheduler decision is whether to recompute context or fetch existing KV from another tier/device.
- The crossover point depends on interconnect and topology (NVLink, PCIe, TCP/RDMA, single-node vs multi-node) and target first-token latency.
- LMCache-style systems use cost models to avoid moving very small chunks inefficiently.

Your original performance question, preserved:

- Sometimes it is cheaper to recompute than to fetch.
- Sometimes it is cheaper to fetch existing KV than to rebuild it.
- The answer depends on topology and transfer path: NVLink, PCIe, TCP, multi-node, single-node, and the amount of data.
- This is especially important if the system is optimizing for first-token latency.

6.11 Tiered storage and key optimizations

- Tiered cache (GPU memory, CPU memory, and colder storage tiers) based on hot/cold access patterns.
- Batched data movement in chunks/pages to improve effective bandwidth.
- Layer-wise pipelining: overlap KV loading with compute.
- Zero-copy style transfers where possible to reduce duplication.
- Asynchronous prefetch for queued requests during idle windows.

Your original key optimizations, rewritten but preserved:

- **Batched data movement:** move chunks/pages instead of many tiny pieces.
- **Layer-wise pipelining:** load cache for the next layer while the current layer computes.

- **Zero-copy direction:** minimize duplicate copies during transfer.
- **Async prefetch:** load likely-needed cache during idle time.
- **Storage hierarchy management:** keep hot context close (GPU/CPU memory), push cold context farther away.

6.12 Quick recall table

Concept	Main benefit	Main tradeoff
KV caching	Avoids redundant prefill compute	Consumes memory capacity
PD disaggregation	Better device specialization/utilization	Extra coordination + transfer overhead
Paged attention	Less fragmentation, better reuse	Added page-management complexity
Tiered cache	Larger effective cache capacity	Eviction/prefetch policy tuning needed

Important

Performance takeaway (from your notes, cleaned):
LMCache-style systems can outperform naive black-box caching pipelines because they are explicitly optimized for I/O patterns, cache placement, and overlap of transfer with compute, even when implemented in Python control layers.

6.13 Video 5: Paged Attention and vLLM-style Serving

Key Idea

Why this matters for inference:
In serving, the real bottleneck is often not just model weights. It is the dynamic KV-cache state that grows with requests and limits throughput.

Your original split between phases, organized:

- **Prefill:** generate KV vectors for the full prompt.
- **Decode:** generate new tokens one at a time using existing KV cache.
- Prefill is compute-heavy and parallel-friendly.
- Decode is much more memory-bound because it must keep and read growing KV state.

System-level implication from your notes:

- Model weights are mostly static memory.
- KV cache is dynamic memory that grows and shrinks with active requests.

- As request count and context length increase, KV cache can become the main factor limiting batch size and throughput.

6.14 Why contiguous KV-cache allocation breaks down

Your original concern, cleaned: many serving systems historically allocated one contiguous chunk of memory per request for KV cache.

- If a system reserves space for future output tokens up front, much of that space may sit unused for a while.
- If requests have different prompt lengths and different generation lengths, the allocator ends up with uneven free gaps.
- This leads to both internal fragmentation and external fragmentation.

Type	What it means	In KV-cache serving
Internal fragmentation	Reserved block is bigger than actually used data	Request gets extra KV space that sits idle until later tokens arrive
External fragmentation	Free memory exists but is split into awkward gaps	System has total free memory, but not in the right shapes for new requests

Added Clarification

OS analogy made explicit:
Paged attention is inspired by virtual memory. Requests behave like processes, KV blocks behave like pages, and the system maps a request's **logical** KV layout to **physical** memory blocks that do not need to be contiguous.

6.15 Paged attention idea

- Instead of one giant contiguous region per request, KV cache is split into fixed-size blocks.
- Logical token order remains contiguous from the model's perspective.
- Physical storage can live in non-contiguous blocks in memory.
- This allows the allocator to fill memory incrementally as requests grow token by token.

What this improves from your notes:

- Less external fragmentation because the system can place blocks wherever space exists.
- Less waste from over-reserving huge contiguous regions.
- Better sharing opportunities between related outputs, such as beam search, parallel sampling, or prefix reuse.

Important

Important clarification:
Paged attention is mainly a **memory-management and serving-systems optimization**. It does not change the Transformer math the way FlashAttention changes how attention is computed on-chip.

6.16 Continuous batching and decode scheduling

Your notes were getting at two batching ideas:

- **Continuous batching:** scheduling decisions are made at each decode iteration, so finished requests leave and new requests can join without waiting for an entire static batch to finish.
- **Mixed batching:** some requests may be in prefill while others are already in decode.

Why this matters:

- Decode runs token by token, so a static batch wastes capacity whenever some requests finish early.
- Continuous batching keeps the GPU busier by filling those open slots quickly.
- Paged attention makes this practical because memory is managed in smaller blocks rather than rigid per-request slabs.

6.17 vLLM-style implementation picture

Your original architecture ideas, organized:

- A central scheduler coordinates which requests run and when.
- A KV-cache manager tracks logical-to-physical block mappings.
- The cache manager knows which blocks are free, allocates new ones, and appends blocks as a request grows.
- Scheduler policy depends on block size, free memory, model length, and whether prefix caching or sliding-window behavior is enabled.

Why this kicked off vLLM in practice:

- It made much better use of HBM during decode.
- It reduced memory waste enough to support larger effective batches and higher serving throughput.
- It exposed the fact that decode throughput is often constrained more by KV-cache management than by raw FLOPs.

6.18 Sharing and branching outputs

- Beam search and parallel sampling create multiple outputs that share a prefix.
- In naive serving, each output may duplicate the same prefix KV cache.
- With block-level management, those shared prefix blocks can be reused instead of copied immediately.
- This is one of the reasons paged/block-based cache management helps much more than just "packing memory better."

6.19 Single-instance vs multi-instance serving

Your original notes moved from one-device systems to fleets. That distinction matters:

- Inside a single serving instance, the scheduler and cache manager directly control GPU memory.
- In a fleet, you need a higher-level orchestrator to decide where requests go and how cache locality is preserved.
- This is where disaggregated serving, routing, and cache transfer become important.

Examples from your notes:

- A prefill server can compute KV cache and send it to a decode server.
- Orchestration above the model server may be done with systems like Kubernetes or Slurm.
- Once you have many instances, ordinary round-robin load balancing is not enough if cache locality matters.

Added Clarification

Session affinity filled in:
Session affinity means routing follow-up requests from the same session/user to the same backend instance whenever possible, so cached state (especially KV cache or conversation context) can be reused locally instead of transferred or recomputed.

6.20 Multi-host serving notes

- A single large model can itself use model parallelism across hosts.
- On top of that, you may also run multiple replicated serving instances behind a load balancer.
- Some features, like speculative decoding, may involve multiple models with different hardware needs.
- In those cases, you care a lot about which path is latency-critical and which path can tolerate extra network hops.

Your original systems intuition, preserved:

- Keep the fast path extremely fast.
- Let the slow or fallback path absorb more latency if necessary.
- Different models may need different hardware and different scaling strategies.

6.21 Final paged-attention memory intuition

- Logical KV order for a request stays clean and contiguous from the model's perspective.
- Physical memory allocation is block-based and can be non-contiguous.
- The last partially filled block can be extended by future tokens of the same request.
- Requests with shared prefixes may share earlier blocks before diverging.

Key Idea

Exam recall line:
FlashAttention is about compute-efficient attention on GPU. Paged attention is about memory-efficient KV-cache management for serving.
